

Guia de Funcionalidades do MarketSystem

Este documento mostra onde mexer nas principais funcionalidades do projeto e destaca exatamente onde configurar o número do WhatsApp.

Projeto localizado em `C:\Users\Mateus_Rodrigues\Desktop\marketsystem`

1. Onde colocar o número do WhatsApp

Principal local para editar: tela admin `/admin/configuracoes`.

1. Abra o sistema localmente.
2. Acesse `/admin/configuracoes`.
3. No campo `WhatsApp`, informe o número da loja.
4. Clique em `Salvar configurações`.

Arquivos envolvidos:

- `src/app/admin/configuracoes/page.tsx`: carrega a tela de configurações.
- `src/components/admin/store-settings-form.tsx`: mostra o campo `WhatsApp` e envia o formulário.
- `src/app/api/store/route.ts`: recebe o `PUT` do formulário.
- `src/services/store-service.ts`: salva o número em `StoreSettings.whatsappNumber`.
- `src/services/order-service.ts`: usa esse número ao criar o link do WhatsApp.

Fallback: se ainda não existir configuração da loja no banco, o sistema também pode usar `NEXT_PUBLIC_WHATSAPP_FALLBACK_NUMBER` no arquivo `.env`. Isso serve como reserva, mas o ideal é salvar o número pela tela admin.

2. Fluxo do pedido

Etapa	Onde mexer	O que faz
Carrinho	<code>src/store/cart-store.ts</code>	Guarda os itens no navegador com Zustand.
Botão adicionar	<code>src/components/cart/add-to-cart-button.tsx</code>	Adiciona o produto ao carrinho.
Checkout	<code>src/components/cart/checkout-form.tsx</code>	Formulário com nome, telefone, entrega/retirada e envio do pedido.
Validação	<code>src/lib/validations.ts</code>	Define o que é obrigatório no pedido.
Criação do pedido	<code>src/services/order-service.ts</code>	Valida estoque, calcula subtotal, taxa, total e gera o WhatsApp.
Mensagem final	<code>src/lib/whatsapp.ts</code>	Monta o texto enviado para a conversa da loja.

3. Configurações da loja

Nome da loja

Campo exibido no topo da loja pública.

```
src/components/admin/store-settings-form.tsx
```

WhatsApp

Número usado para abrir `wa.me` no checkout.

```
src/components/admin/store-settings-form.tsx
```

Taxa de entrega

Aplicada quando o cliente escolhe entrega.

```
src/services/order-service.ts
```

Pedido mínimo

Bloqueia pedidos abaixo do valor definido.

```
src/lib/validations.ts e src/services/order-service.ts
```

Aceita retirada

Libera pedidos sem endereço.

```
src/components/cart/checkout-form.tsx
```

Aceita entrega

Libera pedidos com taxa e endereço.

```
src/components/cart/checkout-form.tsx
```

4. Produtos

- `src/app/admin/produtos/page.tsx` : lista os produtos no admin.
- `src/components/admin/product-form.tsx` : formulário de cadastro/edição.
- `src/services/product-service.ts` : regras de listagem, criação, edição e desativação.
- `src/app/api/products/route.ts` e `src/app/api/products/[id]/route.ts` : endpoints do produto.

Se você quiser alterar campos do produto, como preço, descrição, imagem ou SKU, normalmente vai mexer no formulário do admin e no serviço.

5. Categorias

- `src/app/admin/categorias/page.tsx` : lista categorias.
- `src/components/admin/category-form.tsx` : formulário de categoria.
- `src/services/category-service.ts` : regras de categoria.
- `src/app/api/categories/route.ts` e `src/app/api/categories/[id]/route.ts` : endpoints.

6. Estoque

- `src/components/admin/inventory-form.tsx` : tela para ajustar quantidade e estoque mínimo.
- `src/services/inventory-service.ts` : regra de atualização de estoque.
- `src/app/api/inventory/[productId]/route.ts` : endpoint.

Hoje o sistema consulta o estoque ao criar o pedido. Se quiser baixar estoque automaticamente ao confirmar pedido, essa mudança deve começar em `src/services/order-service.ts`.

7. Pedidos e status

- `src/app/admin/pedidos/page.tsx` : lista de pedidos.
- `src/app/admin/pedidos/[id]/page.tsx` : detalhe do pedido.
- `src/components/admin/order-status-form.tsx` : troca o status.
- `src/services/order-service.ts` : leitura, criação e atualização de status.

Os status disponíveis hoje são: `pending` , `confirmed` , `cancelled` e `delivered` .

8. Área pública

- `src/app/(public)/page.tsx` : home da loja.
- `src/app/(public)/produtos/page.tsx` : catálogo completo.
- `src/app/(public)/categoria/[slug]/page.tsx` : catálogo filtrado por categoria.
- `src/components/product/product-card.tsx` : card visual de cada produto.
- `src/components/layout/site-header.tsx` : cabeçalho da loja.
- `src/components/layout/site-footer.tsx` : rodapé.

9. Banco de dados

O desenho do banco está em `prisma/schema.prisma` .

Modelos principais:

- `Category`
- `Product`
- `Inventory`
- `Order`
- `OrderItem`
- `StoreSettings`

Se você adicionar um campo novo no banco, normalmente precisa mexer em 4 lugares:

1. `prisma/schema.prisma`
2. migration do Prisma
3. `src/lib/validations.ts`
4. o serviço e o formulário da funcionalidade

10. Variáveis de ambiente importantes

Variável	Para que serve
<code>DATABASE_URL</code>	Conexão principal usada pela aplicação.
<code>DIRECT_URL</code>	Conexão usada pelo Prisma para migration e comandos de schema.
<code>NEXT_PUBLIC_APP_URL</code>	URL base da aplicação.
<code>NEXT_PUBLIC_WHATSAPP_FALLBACK_NUMBER</code>	Número reserva caso a loja ainda não tenha um número salvo em <code>StoreSettings</code> .

11. Resumo rápido: onde editar o WhatsApp

1. Preferencialmente na tela `/admin/configuracoes` .
2. O campo visual fica em `src/components/admin/store-settings-form.tsx` .
3. O valor é salvo por `src/services/store-service.ts` .
4. Na criação do pedido, quem usa esse número é `src/services/order-service.ts` .
5. Se precisar de reserva, configure `NEXT_PUBLIC_WHATSAPP_FALLBACK_NUMBER` no `.env` .

Se você quiser, no próximo passo eu também posso criar uma versão 2 deste manual com capturas de tela e setas explicando cada página do admin.